

# Architettura del Database Municipale e Ciclo di Vita del Report (v5.1)

Il sistema implementa un'architettura di persistenza a tre livelli per minimizzare i colli di bottiglia dell'I/O su disco, simulando il comportamento di un motore relazionale a tabelle fisse.

## 1. Mappa dei Livelli di Persistenza e Geometria Hardware

Ogni file all'interno del database adotta una geometria di riga rigida e immutabile. Questo consente l'accesso diretto tramite calcolo matematico dell'offset del cursore, escludendo le scansioni sequenziali orizzontali e le letture orfe.

| Livello / Archivio  | File Fisico Corrispondente                                        | Dimensione Riga         | Meccanismo di Accesso                          | Stato e Indicatori di Cella                                                                                                                                      |
|---------------------|-------------------------------------------------------------------|-------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. RAM Volatile     | Sessione Locale Cittadino                                         | Puntatore Opaco         | $O(1)$ in testa / $O(m)$ ricerca               | Strutture dinamiche mutable ad Information Hiding ( ReportList , ReportStack ).                                                                                  |
| 2. Cache Vettoriale | reports_bench.txt                                                 | REPORT_BENCH_LINE byte) | $O(n)$ sequenziale controllato fino al counter | Buffer circolare statico da 50 slot. Newline invisibile posizionato al byte offset REPORT_BENCH_LINE-1 ( \n ).                                                   |
| 3. Database Master  | open_reports.txt<br>in_progress_reports.txt<br>closed_reports.txt | REPORT_MASTER_LINE Byte | $O(1)$ diretto tramite disk_row                | Struttura stazionaria finale. Byte offset REPORT_MASTER_LINE-2: 'A' (Cella Attiva e Viva), 'N' (Cella Null / Buco vacuo). Byte offset REPORT_MASTER_LINE-1: \n . |

|                             |                             |                                          |                                                      |                                                                                                                          |
|-----------------------------|-----------------------------|------------------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>4. Indice Report ID</b>  | report_AVL_BY_REPORT_ID.txt | <b>22 Byte</b><br>( AVL_REPORT_ID_LINE ) | O(log n)<br>Ricerca Binaria ad array ordinato        | Generato via <i>In-Order</i> AVL. Formato rigidamente compresso:<br>[REPORT_ID(10)]<br>[STATUS(1)]<br>[DISK_ROW(10)]\n . |
| <b>5. Indice User ID</b>    | report_AVL_BY_USER_ID.txt   | <b>21 Byte</b><br>( AVL_USER_ID_LINE )   | O(log n)<br>Ricerca Logaritmica con range bilaterale | Generato via <i>In-Order</i> AVL. Formato rigidamente contratto:<br>[USER_ID(10)]<br>[REPORT_ID(10)]\n .                 |
| <b>6. Registro Centrale</b> | system_total_report.txt     | <b>11 Byte</b><br>( SYSTEM_REG_LINE )    | O(1) diretto su riga fissa tramite fseek             | Memorizza 13 contatori numerici globali atomici in formato binario ( %010u\n ).                                          |

## 2. Flusso Invertito Consistente: Storico del Cittadino

Per garantire l'allineamento assoluto dei dati del cittadino senza consumare risorse in RAM sul server municipale, l'interfaccia esegue la compilazione della vista locale in tempo reale coordinando due distinte fasi sequenziali secondo la regola dell'**Inversione Temporale**:

### [1. FASE MASTER (load\_master\_reports\_to\_list)]

- Il client interroga l'Inorder Array utenti PATH\_AVL\_USER\_ID (21 byte) su disco tramite ricerca binaria in O(log n).
- Raccolto l'intervallo contiguo dei duplicati logici, triangola con l'Inorder Array dei report PATH\_AVL\_REPORT\_ID (22 byte) in O(log n) per ricavare la disk\_row fisica.
- Compie un salto fseek posizionale in O(1) all'interno dei tre canali fisici storici a passi da REPORT\_MASTER\_LINE byte.
- Controllo Cella Attiva:** Il record viene estratto e inserito nella ReportList in RAM solo se il flag al byte offset REPORT\_MASTER\_LINE - 2 è contrassegnato esplicitamente come Active ( 'A' ). Se il flag è 'N' (Null), lo slot viene bypassato poiché la segnalazione è transitata in cache a causa di una modifica recente.

### [2. FASE CACHE BENCH (load\_bench\_reports\_to\_list)]

- Il client scansiona orizzontalmente in O(n) lineare il file transitorio reports\_bench.txt a passi da REPORT\_BENCH\_LINE byte.
- La scansione è rigidamente limitata dal contatore atomico delle immissioni attive della cache ( counter\_bench ).
- Se rileva un report di proprietà del cittadino autenticato, il programma applica un filtro di sbarramento sul suo stato logico interno: se lo stato è diverso da DESTROYED (Stato 3), aggancia direttamente la variante calda della BENCH alla ReportList di sessione.

- Non vi è alcun rischio di collisione o duplicazione con i dati estratti dal Master, poiché i record presenti in cache possiedono una cella Master speculare già invalidata a 'N' e scartata nella prima fase. Se il report in BENCH risulta invece `DESTROYED` (eliminato logicamente dal dipendente), viene interamente deallocato, scomparendo dalla vista dell'utente.

### [3. RENDERING VISIVO E REWIND]

- I report creati ex-novo nella sessione corrente risiedono già in cima alla lista dinamica grazie all'inserimento in testa in  $O(1)$ .
  - L'invocazione finale del metodo `list_rewind` espone la RAM nativa aggiornata dalle sovrascritture, stampando a video lo storico depurato con i dati aggiornati all'ultimo millisecondo.
- 

## 3. Algoritmo di Sfoltimento ed Eliminazione del Dipendente

Quando il dipendente comunale utilizza la funzione `employee_change_report_status` per far avanzare una pratica o distruggerla ( `DESTROYED` ), il finto server applica un algoritmo di sfoltimento asincrono coordinato al singolo byte che esclude le scansioni sequenziali orizzontali dei file storici:

1. **Ricerca Orizzontale in Bench/Cache:** Il sistema verifica se il report risiede momentaneamente nella BENCH in  $O(n)$ . Se presente, aggiorna i contatori nel registro in  $O(1)$  tramite la funzione helper `update_system_counters`, applica il nuovo stato e rispiana lo slot della cache a `REPORT_BENCH_LINE` byte scrivendo il record modificato.
  2. **Ricerca Logaritmica e Invalidazione Master:** Se assente in BENCH, interroga l'array d'indice dei report su disco in  $O(\log n)$  tramite la funzione logaritmica `findReportId`. Individuata la `disk_row` fisica, il server esegue un salto `fseek` in  $O(1)$  sul file Master di provenienza e **sovrascrive il byte offset `REPORT_MASTER_LINE - 2` impostandolo a 'N' (Null)**, distruggendo l'attivazione della cella e trasformandola in un buco vuoto.
  3. **Accatastamento LIFO dei Buchi:** L'indice della riga master appena liberata viene immediatamente spinto (operazione Push in  $O(1)$ ) in coda al file ausiliario dello stack dei buchi specifico ( `open_holes.txt`, ecc.), occupando una riga rigida da 11 byte formattata come `%010d\n`.
  4. **Immissione Controllata in Cache:** Il report modificato viene caricato in Append in fondo alla BENCH a passo `REPORT_BENCH_LINE` byte, portando il contatore della cache a +1, ma **senza mai incrementare il numero dei report totali del Comune**, in quanto la segnalazione era già censita nel sistema. Se il contatore tocca la costante limite ( `LIMIT_BENCH` ), innesca in automatico la funzione `process_and_flush_bench()`, la quale scarica la cache estraendo e riciclando chirurgicamente i buchi LIFO in  $O(1)$ , e se lo stack dei buchi restituisce `-1` (vuoto), carica il record in Append alla coda fisica del file di stato calcolando l'offset tramite la dimensione reale del file system.
- 

## 4. Validazione e Pipeline di Debugging del Sistema

A causa dei vincoli temporali stringenti indotti dalla preparazione di più esami della sessione universitaria, l'architettura software è stata validata e blindata escludendo stress-test su scale dimensionali variabili e concentrando l'intera fase di debugging sulla configurazione fissa stazionata nel file `config.h` ( `LIMIT_BENCH = 2`, `MAX_USERNAME = 50`, `MAX_PASSWORD = 50`, `MAX_DESC = 256`, `BLOCK_SIZE_USERS = 50` ).

Il corretto funzionamento del server e dei meccanismi di I/O binario è stato pienamente convalidato ed è andato a buon fine impostando la suite di test integrata all'inizio dell'esecuzione (Opzione 3) e applicando la rigorosa Road Map a fasi.

I test isolati verificano l'allineamento delle asimmetrie (REPORT\_BENCH\_LINE byte Bench vs REPORT\_MASTER\_LINE byte Master) e l'interruzione nativa della lettura dei blocchi mediante il controllo del valore di ritorno di `fread`, che si arresta in modalità autonoma non appena rileva l'EOF hardware, escludendo l'uso di righe sentinella orfane che causerebbero sfasamenti prestazionali. Al termine dell'esecuzione dei test, lo stato iniziale prefissato del database viene ripristinato piattando i file transitori e ri-inizializzando il registro statistico statico `system_total_report.txt` al blocco vergine di 143 caratteri (13 righe da 11 byte di zeri).